

KRR Practical: Sorting Algorithms (Bubble + Selection Sort)

Rule: IF $A > B$ THEN Swap(A,B)

```
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(len(arr)-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

def selection_sort(arr):
    for i in range(len(arr)):
        m=i
        for j in range(i+1,len(arr)):
            if arr[j] < arr[m]:
                m=j
        arr[i], arr[m] = arr[m], arr[i]
    return arr

numbers=[64,25,12,22,11]
print("Original:", numbers)
print("Bubble:", bubble_sort(numbers.copy()))
print("Selection:", selection_sort(numbers.copy()))
print("Complexity:  $O(n^2)$ , Space:  $O(1)$ ")
```

Output: [11,12,22,25,64]

NLP Information Retrieval System (NLTK) - Final Code

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.probability import FreqDist
from nltk import pos_tag

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

text="Natural Language Processing helps computers understand human language."
tokens = word_tokenize(text)
print("Tokens:", tokens)

freq = FreqDist(tokens)
for word,count in freq.items():
    print(word,":",count)

stop_words=set(stopwords.words('english'))
filtered=[w for w in tokens if w.lower() not in stop_words]
print("Filtered:", filtered)

pos_tags = pos_tag(filtered)
print("POS Tags:", pos_tags)
```

Travelling Salesperson Problem (TSP) - Heuristic Search (Nearest Neighbor)

```
# Cost matrix between cities
cost = [[0,10,15,20],
        [10,0,35,25],
        [15,35,0,30],
        [20,25,30,0]]

n = len(cost)
visited = [0]      # Start city
total = 0          # Total cost

while len(visited) < n:
    city = visited[-1]

    nxt = min(
        (cost[city][i], i)
        for i in range(n)
        if i not in visited
    )

    total += nxt[0]
    visited.append(nxt[1])

# Return to starting city
total += cost[visited[-1]][0]
visited.append(0)

print("Path:", visited)
print("Minimum Cost:", total)
Output: Path=[0,1,3,2,0], Minimum Cost=80
```

Water Jug Problem using BFS (Python)

Simple code for AI practical examination

```
from collections import deque

jug1 = int(input("Enter capacity of jug1: "))
jug2 = int(input("Enter capacity of jug2: "))
target = int(input("Enter target amount: "))

visited = set()
queue = deque()
queue.append((0,0))

while queue:
    x, y = queue.popleft()

    if (x,y) in visited:
        continue

    visited.add((x,y))
    print((x,y))

    if x == target or y == target:
        print("Target achieved")
        break

    queue.append((jug1,y))
    queue.append((x,jug2))

    queue.append((0,y))
    queue.append((x,0))

    transfer = min(x, jug2-y)
    queue.append((x-transfer, y+transfer))

    transfer = min(y, jug1-x)
    queue.append((x+transfer, y-transfer))
```

Tic-Tac-Toe (Human vs Computer) - Simple Python Code

```
# Create empty board
board = [" "] * 9

def show():
    print(board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(board[6], "|", board[7], "|", board[8])

def check(player):
    win=[(0,1,2),(3,4,5),(6,7,8),
        (0,3,6),(1,4,7),(2,5,8),
        (0,4,8),(2,4,6)]
    for x in win:
        if board[x[0]]==board[x[1]]==board[x[2]]==player:
            return True
    return False

for i in range(9):
    show()
    move=int(input("Enter position (1-9): ")) - 1
    board[move]="X"

    if check("X"):
        show()
        print("Human Wins!")
        break

    if " " not in board:
        show()
        print("Draw!")
        break

    for j in range(9):
        if board[j]==" ":
            board[j]="O"
            break

    if check("O"):
        show()
        print("Computer Wins!")
        break
```

Positions: 1|2|3 / 4|5|6 / 7|8|9